AutaMedica — Arquitectura del Sistema

Versión: 2026-03-28 | **Repo:** `ecucondorSA/Autamedica` | **Stack:** Astro 5 + React 19 + Supabase + Cloudflare

---

1. Visión General

AutaMedica es una plataforma de telemedicina construida como **monorepo** con Turborepo + pnpm. Un solo build se despliega a **4 subdominios** en Cloudflare Pages, cada uno sirviendo un portal diferente detectado por hostname en el middleware.

Subdominio	Proyecto CF Pages	Portal
`www.autamedica.com`	`autamedica-web`	Landing público + auth
`doctor.autamedica.com`	`autamedica-doctors`	Portal médico
`paciente.autamedica.com`	`autamedica-patients`	Portal paciente
`empresa.autamedica.com`	`autamedica-companies`	Portal empresa

Métricas del codebase: 3,761 archivos TS/TSX · 54 endpoints API · 50+ tablas DB · 4 edge functions

2. Estructura del Monorepo

Apps (6)

App	Tecnología	Función
`apps/web`	Astro 5 + React 19	App principal (multi-subdomain)
`apps/signaling-server`	Cloudflare Worker + Durable Objects	Señalización WebRTC
`apps/clinical-ai-orchestrator`	Worker	Audio STT → Clinical AI → JSON
`apps/reminder-worker`	Bun daemon	Recordatorios WhatsApp
`apps/android`	Kotlin + Jetpack Compose	App móvil nativa
`apps/mobile`	—	Reservado para convergencia iOS/Android

Packages (11)

Package	Descripción
`@autamedica/types`	190+ tipos TypeScript, branded IDs (PatientId, DoctorId), Zod schemas
`@autamedica/shared`	Logger, env validation, role routing, utilidades
`@autamedica/auth`	Supabase SSR client, session hooks, roles
`@autamedica/hooks`	Hooks React de dominio médico
`@autamedica/telemedicine`	Motor WebRTC completo (74 archivos)
`@autamedica/ui`	Componentes UI compartidos
`@autamedica/tailwind-config`	Preset Tailwind CSS
`@autamedica/scheduling`	Tipos de turnos, disponibilidad, recordatorios
`@autamedica/session`	State management de sesión
`@autamedica/supabase-client`	Factory de clientes Supabase
`@autamedica/typescript-config`	Preset de TSConfig

3. Arquitectura de Capas

Frontend (4 capas)

```
UI / View  →  Hook / Controller  →  Service  →  API Client  →  [HTTP]
```

Capa	Ubicación	Responsabilidad
UI / View	`components/*.tsx` / `*.astro`	Renderiza. Sin lógica de negocio, sin fetch.
Hook / Controller	`hooks/*.ts`	Estado local. Consume services. Max ~50 líneas.
Service (frontend)	`lib/*/services/*.ts`	Lógica de negocio pura. Sin estado React.
API Client	`lib/api-client.ts`	fetchApi() para endpoints Astro. Centraliza auth.

Backend (3 capas)

```
Controller (APIRoute)  →  Service  →  Repository  →  [Supabase/DB]
```

Capa	Ubicación	Responsabilidad
Controller	`pages/api/*.ts`	Recibe request, valida, delega. Max ~30 líneas.
Service (backend)	`lib/*/services/*.backend.ts`	Reglas de negocio, orquestación.
Repository	`lib/*/repos/*.ts`	Única capa que habla con Supabase. CRUD tipado.

Anti-patrones Prohibidos

- × **God Object Hook**: Hook con estado + lógica + fetch
- × **Fat Controller**: API route con queries SQL y lógica
- × **Supabase en componentes**: Usar hooks de `@autamedica/auth`
- × **Lógica de negocio en UI**: Si tiene if/else de dominio → Service

4. Motor de Telemedicina (74 archivos)

Core WebRTC

Archivo	Líneas	Responsabilidad
`webrtc-client.ts`	699	Orquestador: lifecycle, signaling, peers, events
`media-manager.ts`	287	Captura cámara/mic, devices, tracks, codecs
`quality-monitor.ts`	474	Métricas: bitrate, latency, packet loss, MOS
`bitrate-adapter.ts`	212	Rampa progresiva de bitrate por calidad de red
`peer-connection-factory.ts`	275	Creación de RTCPeerConnection + event handlers
`signaling-socket.ts`	114	WebSocket lifecycle, message routing
`turn-manager.ts`	54	Refresh de credenciales TURN (Cloudflare, 24h)
`media-optimizations.ts`	212	Constraints adaptativos, content hints

Hooks React (15)

Hook	Función
`useTelemedicineClient`	Lifecycle del client WebRTC
`useVideoCallReducer`	Consolidación de 6+ useState
`useCallDuration`	Timer + formato display
`useCallQuality`	Monitoreo + sugerencia audio-only
`useGreenRoom`	Preview pre-llamada, VU meter
`useChatMessages`	Chat: estado, envío, scroll
`useScreenShare`	Toggle de compartir pantalla
`useCallStatusPersistence`	Sync estado → DB
`useCallLifecycleEffects`	Wake lock, visibility, streams
`useAutoHideControls`	Auto-ocultar tras inactividad
`useParticipantSound`	Sonido de entrada (AudioContext)
`useViewportDetection`	Desktop vs mobile
`useCallDebugger`	Debug events
`useRemoteMediaTimeout`	Timeout sin media remota
`useMediaControls`	Mute/unmute, cámara on/off

Componentes UI (7)

`ActiveCallLayout` · `CallControlsBar` · `CallTopBar` · `GreenRoom` · `ChatPanel` · `QualityOverlays` · `ConnectingOverlay`

Flujo de una Videollamada

```
Doctor clickea "Llamar" → start-call (Edge Function) → DB: calls.status = 'requested'
  → Paciente recibe notificación → Acepta → DB: status = 'accepted'
  → Ambos: GET /api/calls/[roomId]/bootstrap → ICE servers + TURN credentials
  → WebSocket signaling → SDP offer/answer → ICE candidates
  → RTCPeerConnection established → Media streams flowing
  → QualityMonitor polls → BitrateAdapter adjusts
  → Call ends → DB: status = 'ended' → Post-consultation report
```

5. Portal del Doctor (40 componentes)

Estructura

Archivo	Líneas	Responsabilidad
`DoctorsHomePage.tsx`	586	Dashboard: stats, turnos, acciones rápidas
`QuickActionsPanel.tsx`	169	Sidebar: notas, recetas, IA
`DiagnosisPanel.tsx`	74	Modal de diagnóstico con IA
`VideoControlsOverlay.tsx`	102	Barra de controles de video
`DoctorsPortalShell.tsx`	685	Shell de navegación
`SidebarNavigation.tsx`	80	Menú lateral
`DoctorCallPage.tsx`	559	UI de videollamada

Hooks del Doctor (19)

`useVideoCallControls` · `useClock` · `useSidebar` · `useAuthenticatedUser` · `useDoctorStats` · `useDoctorAppointments` · `useConsultationContext` · `useStartCall` · `useRealPatients` · `useAIAnalysis` + 9 más

6. Portal del Paciente (61 componentes)

Estructura

Archivo	Líneas	Responsabilidad
`AnamnesisPage.tsx`	690	Formulario de anamnesis (intake)
`VersionTimeline.tsx`	100	Timeline de versiones
`PrivacyAcceptanceScreen.tsx`	86	Aceptación de privacidad
`AnalysisResultOverlay.tsx`	91	Resultados de análisis IA
`PatientProfilePage.tsx`	675	Perfil del paciente
`AppointmentsPage.tsx`	—	Agenda de turnos
`CommunityPage.tsx`	696	Foro de comunidad

Componentes Compartidos

Archivo	Función
`FormFields.tsx`	`Field` + `SelectField` reutilizables
`MetricCard.tsx`	Tarjeta de métrica
`CrossModuleCard.tsx`	Tarjeta cross-módulo con stats

Hooks del Paciente (28)

`useAppointments` · `useAnamnesis` · `useCommunity` · `useMedicalChat` · `useProfile` · `usePatientReports` · `useVideoCall` · `useTelemedicineSignaling` · `useMedicalHistory` · `usePatientProfilePage` + 18 más

7. Lógica Server (Backend)

Server Paciente (`lib/patients/server/` — 8 archivos)

Archivo	Líneas	Función
`patient-insights.ts`	390	Orquestador: tendencias, risk flags
`normalize.ts`	221	10 normalizadores de datos
`alert-engine.ts`	338	Alertas determinísticas + fingerprinting
`dashboard-builder.ts`	180	Dashboard summary + prioridades
`anamnesis-ai-extraction.ts`	578	Extracción de entidades clínicas via IA
`ai-provider.ts`	107	Resolución OpenAI / Gemini
`extraction-parser.ts`	102	Parser IA → formato estructurado
`body-selection.ts`	38	Dedup de selecciones en diagrama corporal

Servicios Frontend (`lib/patients/services/` — 20 archivos)

Archivo	Función
`pdf-report.service.ts` (684 lín)	Generación de PDF de consulta
`pdf-report-config.ts` (75 lín)	Constantes del PDF
`profile-form.ts` (299 lín)	13 helpers puros de formulario
`medical-history.frontend.ts`	Transformaciones para UI
`anamnesis.service.ts`	CRUD anamnesis, secciones
`appointments.service.ts`	Turnos: crear, cancelar, reprogramar
`community.service.ts`	Posts, comentarios, moderación

8. Infraestructura Supabase

Proyecto: `qxjjrpksmwwrmubhvjew` (sa-east-1)

Edge Functions (4)

Función	Trigger	Acción
`create-call`	POST desde doctor	Crear registro de llamada
`update-call-status`	PATCH	Actualizar estado, registrar duración
`ingest-patient-observation`	POST	Signos vitales + alertas críticas
`send-push-notification`	POST	Firebase Cloud Messaging

Migraciones Recientes (20)

Últimas 5 significativas:

- `create_patient_community.sql` — Foro de pacientes
- `create_anamnesis_versions.sql` — Versionado de anamnesis
- `create_anamnesis_extractions.sql` — Registros de extracción IA
- `harden_call_status_transitions.sql` — Validación de estados de llamada
- `harden_referential_integrity.sql` — Integridad referencial

Roles y Portales

Rol	Portal	Ruta base
`patient`	paciente.autamedica.com	`/dashboard`
`doctor`	doctor.autamedica.com	`/doctor/dashboard`
`company`	empresa.autamedica.com	`/company/dashboard`
`admin`	admin.autamedica.com	`/admin/dashboard`

9. API Endpoints (54 totales)

Doctor Portal (20 endpoints)

- `GET/PUT /api/doctors/profile` — Perfil del doctor
- `GET /api/doctors/ranking` — Leaderboard
- `POST /api/doctors/calls/start` — Iniciar llamada
- `GET /api/doctors/consultations/[roomId]/context` — Datos del paciente en consulta
- `POST /api/doctors/consultations/[roomId]/report` — Generar informe
- `GET /api/doctors/patients` — Lista de pacientes
- `GET /api/doctors/patients/[id]/anamnesis` — Anamnesis del paciente
- `GET /api/doctors/turn-credentials` — Credenciales TURN

Patient Portal (18 endpoints)

- `GET/POST /api/patients/appointments` — Turnos
- `GET /api/patients/medical-history` — Historia clínica
- `POST /api/patients/anamnesis/extract` — Extracción IA
- `GET /api/patients/anamnesis/pdf` — Generar PDF
- `PUT /api/patients/profile/update` — Actualizar perfil

Shared (11 endpoints)

- `GET /api/calls/[roomId]/bootstrap` — Inicializar llamada (ambos portales)
- `GET /api/calls/[callId]/status` — Estado de llamada
- `GET /api/health` — Health check

10. Detección de Portal (Middleware)

El middleware de Astro (424 líneas) detecta el portal según el contexto:

Producción: Por hostname → `doctor.autamedica.com` renderiza portal médico

Desarrollo: Por query param → `localhost:3000?portal=doctor`

Flujo del middleware:

1. Detectar portal (hostname o `?portal=`)
2. Si ruta protegida → verificar sesión Supabase
3. Si usuario con rol → verificar que está en su portal correcto
4. Si ruta legacy (`/empresa`, `/médico`) → redirect 301
5. Inyectar `locals.session` y `locals.supabase` para API routes

Cookies Cross-Subdomain

Las cookies de Supabase se setean con `domain='.autamedica.com'` para ser visibles en todos los subdominios. Sin este domain, un login en `www` no se ve desde `doctor`.

11. Reglas de Código

Límites Hard

- **<700 líneas** por archivo de código fuente
- **0 errores + 0 warnings** en `pnpm lint`
- **0 `any`** — usar `unknown` + type guards
- **0 `TODO/FIXME/HACK`** — resolver o escalar
- **0 datos hardcodeados** — todo viene del backend

Build & Deploy

```
git push origin main
  → CI: typecheck + lint
  → DB Migrations (si hay)
  → Build Astro → dist/
  → Deploy dist/ a 4 CF Pages projects
  → Smoke tests (200 en homepage)
  → Auto-rollback si falla
```

Comandos Esenciales

```
pnpm build:packages:core    # Compilar packages
pnpm lint                   # ESLint monorepo
pnpm typecheck              # TypeScript check
pnpm --filter @autamedica/web check  # Astro check
pnpm db:sync                # Regenerar tipos DB
```

Documento generado el 2026-03-28. Fuente: CLAUDE.md + análisis estático del codebase.